UNIT 4:IN-DEPTH MALWARE ANALYSIS

Dhanashree Kulkarni

Index

- Recognizing Packed Malware;
- Getting Started with Unpacking;
- Using Debuggers for Dumping Packed Malware from Memory;
- Analyzing Multi-Technology and Fileless Malware;
- Code Injection and API Hooking;
- Using Memory Forensics for Malware Analysis;
- JavaScript DE obfuscation;
- PDF Document Analysis;
- Office Document Analysis.

Recognizing Packed Malware

- Malware is created with **deception** in mind.
- Malware authors want to go **undetected** in order to **steal, alter or delete as much information as possible.**
- **Obfuscating** malware is a way to keep the files associated with the malware from detection and easy analysis.
  - Obfuscation **takes code** and makes it **unreadable without destroying its intended functionality**.
  - This technique is used to **delay detection** and/or **to make reverse engineering difficult**. Obfuscation does have legitimate purpose.
  - It can be used to protect intellectual property or other sensitive code.
- **Packing** is a type of obfuscation technique.

What is obfuscation?

- Making anything **tough** to **grasp** is referred to as **obfuscation.**
- To **protect intellectual property** or **trade secrets,** and to **prevent** an **adversary** from **reverse engineering** a **proprietary software** application, programming code is **frequently obfuscated.**
- One form of obfuscation is to **encrypt some or all of a program's code.** An "obfuscator" is a tool that converts simple source code into a program that does the same thing but is more difficult to read and understand.

Malware obfuscation:

- Malware authors often use packing or obfuscation technique to make their files more difficult to detect or analyse.
- **Malware obfuscation is a technique used to create textual and binary data difficult to interpret.** It helps adversaries to **hide critical strings in a program,** because they reveal patterns of the malware's behavior. The strings would be registry keys and infected URLs.
- **Packed programs are a subset of obfuscated programs in which the malicious program is compressed and cannot be analyzed.** Packer and obfuscation techniques will limit the attempts to statically analyse the malware.
- **Non-Malicious programs always include many strings. Malware that is packed or obfuscated contains very few strings.** If the program has only few strings, it is probably either obfuscated or packed, which gives a clue that it may be malicious.

Malware Obfuscation Techniques:

- Since malware writers frequently employ obfuscation to evade antivirus scanners, it's important to understand how this technique is used in malware. Here we have a few Obfuscation Techniques which is used to pack the malicious strings:

1-Dead-Code Insertion:

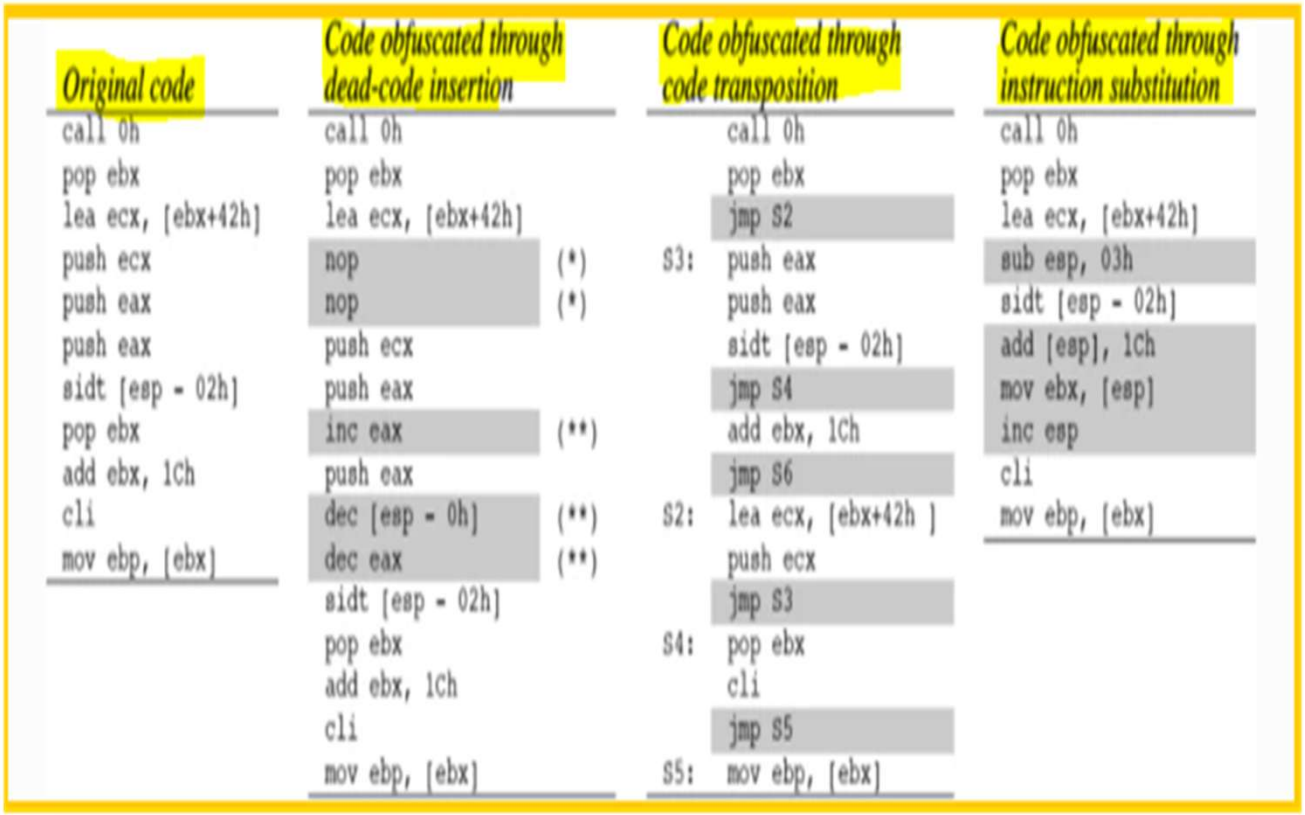
- A dead-code insertion is a simple approach for **changing the appearance of a program while maintaining its functionality**. NOP is an example of such a command. The original code is easily obfuscated by inserting NOP instructions. Signature-based antivirus scanners, on the other hand, can resist this strategy by simply removing the unsuccessful instructions before analyzing them.
- NOP is typically used to generate a delay in execution or to reserve space in code memory. Execution continues with the next instruction. No registers or flags are affected by this instruction. NOP is typically used to generate a delay in execution or to reserve space in code memory.

- 2-XOR:
- This popular method of obfuscation conceals data so it cannot be analyzed.
- Applying XOR twice with the same key restores the original value.
- It does this by **swapping the contents of two variables inside the code,** such as:
  - XOR EBX, EAX
  - XOR EAX, EBX
  - XOR EBX, EAX

- 3-Register reassignment:
  - Register reassignment is another simple technique that switches registers from generation to generation (malware generation) while keeping the program code and its behavior the same.
  - Note that wildcard searching can make this technique useless.
- 4-Subroutine Reordering:
  - Subroutine reordering obfuscates an original code by randomly rearranging its subroutines. This method can generate n! different variations, where n denotes the number of subroutines.

- 5-Instruction Substitution:
  - Instruction substitution evolves an **original code by replacing some instructions with other equivalent ones.** This technique can effectively change the code with a **library of equivalent instructions.**
- 6-Code Transposition:
  - Code transposition **reorders the sequence of instructions in an original code without affecting the behavior of the code.** This procedure can be accomplished in **two ways:**
    - The first technique **shuffles the instructions at random, then inserts unconditional branches or jumps to restore the original execution order.** It is not difficult to defeat this method because the original program can be easily restored by removing the unconditional branches or jumps.
    - The second method **creates new generations by choosing and reordering the independent instructions that have no impact on one another.** Because it is a complex problem to find the independent instructions, this method is hard to implement, but can make the cost of detection high.

- 7-Code integration:
- Code integration was first spotted in the Zmist/Win95 virus (also known as Zmist), and it instructs malicious code to knit itself to the target program's code.
- The malware **decompiles the program into manageable bits, inserts itself between them, and then reassembles the injected code into a new variant to use the technique.**



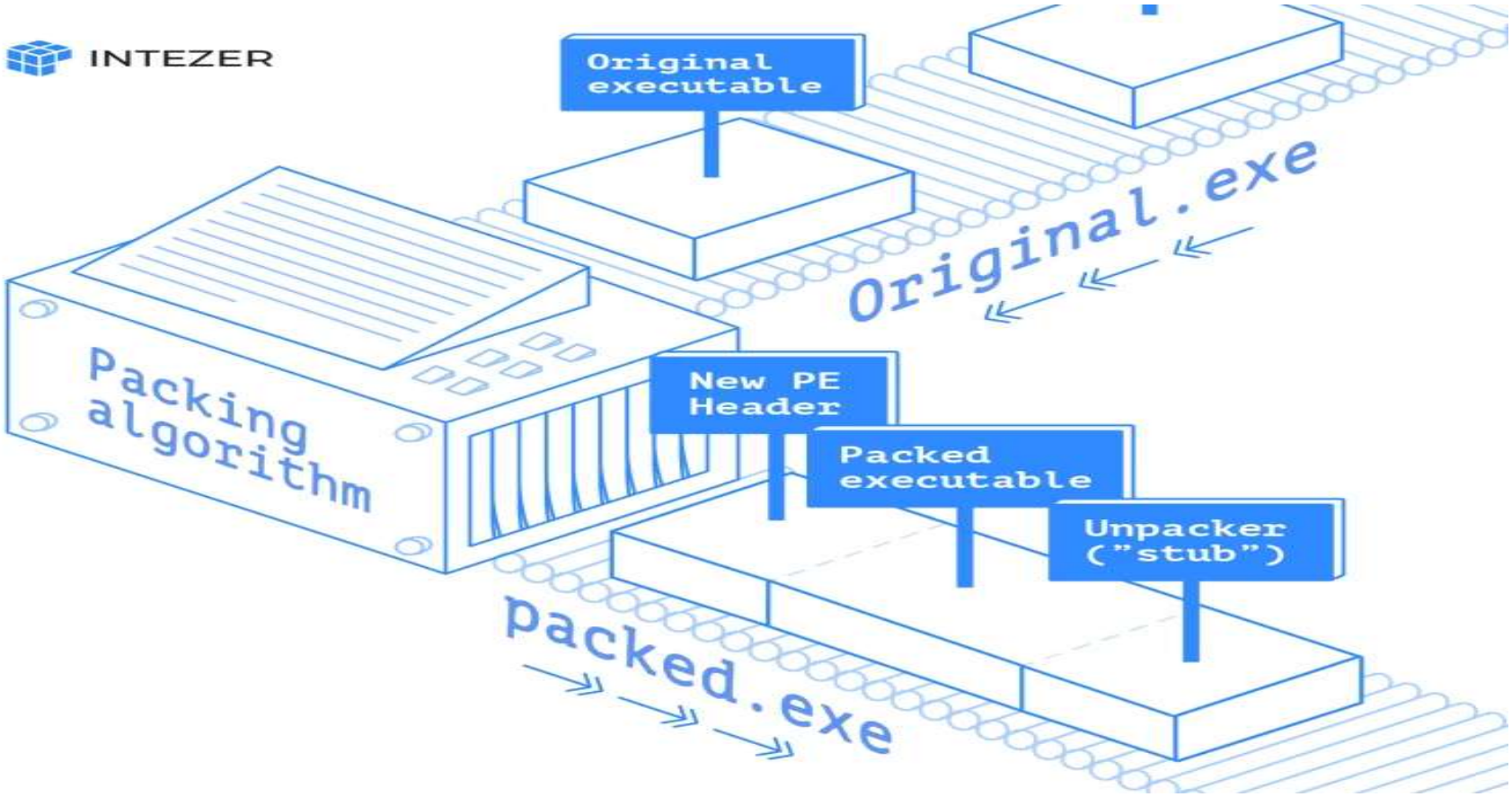
Original code	Code obfuscated through dead-code insertion		Code obfuscated through code transposition		Code obfuscated through instruction substitution
call 0h	call 0h		call 0h		call 0h
pop ebx	pop ebx		pop ebx		pop ebx
lea ecx, [ebx+42h]	lea ecx, [ebx+42h]		jmp S2		lea ecx, [ebx+42h]
push ecx	nop	(*)	S3: push eax		sub esp, 03h
push eax	nop	(*)	push eax		sidt [esp - 02h]
push eax	push ecx		sidt [esp - 02h]		add [esp], 1Ch
sidt [esp - 02h]	push eax		jmp S4		mov ebx, [esp]
pop ebx	inc eax	(**)	add ebx, 1Ch		inc esp
add ebx, 1Ch	push eax		jmp S6		cli
cli	dec [esp - 0h]	(**)	S2: lea ecx, [ebx+42h]		mov ebp, [ebx]
mov ebp, [ebx]	dec eax	(**)	push ecx		
	sidt [esp - 02h]		jmp S3		
	pop ebx		S4: pop ebx		
	add ebx, 1Ch		cli		
	cli		jmp S5		
	mov ebp, [ebx]		S5: mov ebp, [ebx]		

- 8-Instruction Substitution:
  - Instruction substitution evolves **an original code by replacing some instructions with other equivalent ones**. This technique can effectively change the code with a library of equivalent instructions.
  - Eg: ADD EAX, 1 substituted with INC EAX
- 9-Base64:
  - Base64 is another well-known obfuscation technique used by adversaries.
  - It's essentially a 64-character encoding scheme, with the **padding character** being the **=** **(equal) sign**. The alphabet also includes the letters a-z, A-Z, + and /, and 0-9 characters.
- How it works:
  - Take 3 bytes (24 bits) of data
  - Divide them into four groups of 6 bits
  - Each 6-bit group is mapped to a Base64 character

- 10-Packers:
  - In some situations, the entire program is obfuscated to prevent the malware code from being detected until it is placed into memory.
  - This is accomplished with the aid of software that compresses an executable to make it smaller.
  - The compressed executable is then packaged inside the code required to decompress itself during runtime.
  - The decompression procedure assures that the executive file does not resemble its original state.

Packing

- Packing is a subset of obfuscation.
- A packer is a tool that modifies the formatting of code by compressing or encrypting the data.
- Though often used to delay the detection of malicious code, there is still legitimate use for packing.
- Some legitimate use includes protecting intellectual property or other sensitive data from being copied.
- A stub is a small portion of code that contains the decryption or decompression agent used to decrypt the packed file


INTEZER
Original executable
Original.exe
Packing algorithm
New PE Header
Packed executable
Unpacker ("stub")
packed.exe

The packing process consists of:

1. The original code is uploaded into the packer tool and goes through the packer process to compress or encrypt the data
2. The original portable executable header (PE header, which consists of executable image and object files) and original code are compressed or encrypted and stored in the packed section of the new executable
3. The packed file consists of:
   - New PE header
   - Packed section(s)
   - Decompression stub — used to unpack the code
4. During the packing process, the **original entry point is relocated/obfuscated in the packed section.** This is important for anyone trying to analyze the code. This process makes identifying the import address table (IAT) and original entry point difficult
5. The decompression stub is used to unpack the code upon delivery

- Some malware creators use custom packers, but commercial/open-source packers are also used. Some popular packers include:
  - UPX
  - Themida
  - The Enigma Protector
  - VMProtect
  - Obsidium
  - MPRESS
  - Exe Packer 2.300
  - ExeStealth

How to detect packed files

- Static detection
  - When data is being compressed or encrypted, the output is not structured and resembles more random data, **resulting in high entropy**. In information theory, **entropy is a measure of disorder/randomness.** The calculation is done using **logarithm with base 2**, also known as Shannon entropy, the result is in units of bits, hence the range of entropy level for binary files is 0 to 8.
  - We can check the entropy of the file sections, and if one of them has a **value of *7* or higher, there is a high probability that the file is packed.**
  - Packers may change the final payload's section names.
  - Multiple tools can be used to detect indicators of a sample being packed:
    - PEstudio, DiE (Detect it Easy), CFF explorer, and more.

- Dynamic detection
  - When we dynamically analyze a packed file, we **aim to extract the payload**. Several functions can be a good place for putting a breakpoint and attempting to fetch the extraction process. Let's look at API calls and debuggers.
  - API Calls
  - Essentially, all packers need to perform the following operations:
    - allocate memory, change permissions, read the encrypted/packed chunk of code, decrypt it, load it to the allocated memory space and execute it.
    - To make all of these actions, the **program must use system calls because these actions require interaction with the kernel**.

- The relevant system calls are:
- VirtualAlloc:
  - used for allocating memory in the current process. The memory is automatically initialized to zero. One of the arguments is dwSize – that's the size in bytes. Knowing the size can help identify when a program allocates big memory space, possibly for the unpacked payload.
- VirtualProtect:
  - changes the permissions of the given virtual address. When malware wants to grant write or execute permissions flNewProtect argument will be set to PAGE_EXECUTE_READWRITE (0x40) or PAGE_EXECUTE_READ (0x20).
- RtlDecompressBuffer: decompresses the provided buffer.
- CreateProcessInternalW: creates a new process. In the context of malware execution, it can create a new threat with the malicious unpacked payload.
- We can look for these system calls in the list of imported functions. However, **malware authors usually try to hide these system calls**. For instance, we will not see them in the imported function list, and they not even be part of the strings in the sample. In this case, the function and the corresponding library are dynamically loaded at runtime – also known as explicit linking.
- In this case, dynamic debugging can be very helpful – debuggers like x64dbg let you put a breakpoint on function calls, even if the function is not imported. The breakpoint will be triggered if the system call is executed.

Using Debuggers for Dumping Packed Malware from Memory

- A debugger is a powerful tool that allows developers and researchers to follow and control the execution of a program. When debugging an executable, you can view the registers, stack, and memory and see how each instruction affects the stored data.
- Debugging malware can reveal code executed only in runtime (meaning that you will not see it in standard static analysis). Debuggers also allow you to see how strings and payloads are deobfuscated and constructed – making it easier to find the interesting patterns of the malware.
- Different debuggers can be used to debug Windows executables and DLLs:
  - OllyDbg – not maintained anymore
  - Windbg – powerful kernel-mode and user-mode debugger created by Microsoft.
  - x64dbg – reliable and user-friendly. For 32-bit executables, it's x32dbg.
- The main difference between these debuggers is the implementation, which may result in certain behaviors, but for the most part, it's up to you to choose which debugger you prefer.
- Example : https://infosecwriteups.com/extracting-packer-injected-malware-from-memory-remcos-rat-aa87cb224b70

Analyzing Multi-Technology and Fileless Malware

- What is Fileless Malware?
- Fileless malware is a type of malicious activity that **uses native, legitimate tools built into a system to execute a cyber attack.** Unlike traditional malware, fileless malware **does not require an attacker to install any code on a target's system, making it hard to detect.**
- This fileless technique of **using native tools to conduct a malicious attack** is sometimes referred to as **living off the land** or **LOLbins**
- While attackers don't have to install code to launch a fileless malware attack, **they still need to get access to the environment so they can modify its native tools to serve their purposes.** Access and attacks can be accomplished in several ways, such as through the use of:
  - Exploit kits
  - Hijacked native tools
  - Registry resident malware
  - Memory-only malware
  - Fileless ransomware
  - Stolen credentials

Stages of a fileless attack

- Stage #1: Gain Access
  - Technique: Remotely Exploit a vulnerability and use web scripting for remote access (eg. China Chopper)
  - The attacker gains remote access to the victim's system, to establish a beachhead for his attack.
- Stage #2: Steal Credentials
  - Technique: Remotely Exploit a vulnerability and use web scripting for remote access (eg. Mimikatz)
  - Using the access gained in the previous step, the attacker now tries to obtain credentials for the environment he has compromised, allowing him to easily move to other systems in that environment.
- Stage #3: Maintain Persistence
  - Technique: Modify registry to create a backdoor (eg. Sticky Keys Bypass)
  - Now, the attacker sets up a backdoor that will allow him to return to this environment at will, without having to repeat the initial steps of the attack.
- Stage #4: Exfiltrate Data
  - Technique: Uses file system and built-in compression utility to gather data , then uses FTP to upload the data
  - In the final step, the attacker gathers the data he wants and prepares it for exfiltration, copying it in one location and then compressing it using readily available system tools such as Compact. The attacker then removes the data from the victim's environment by uploading it via FTP.

How to Detect Fileless Malware

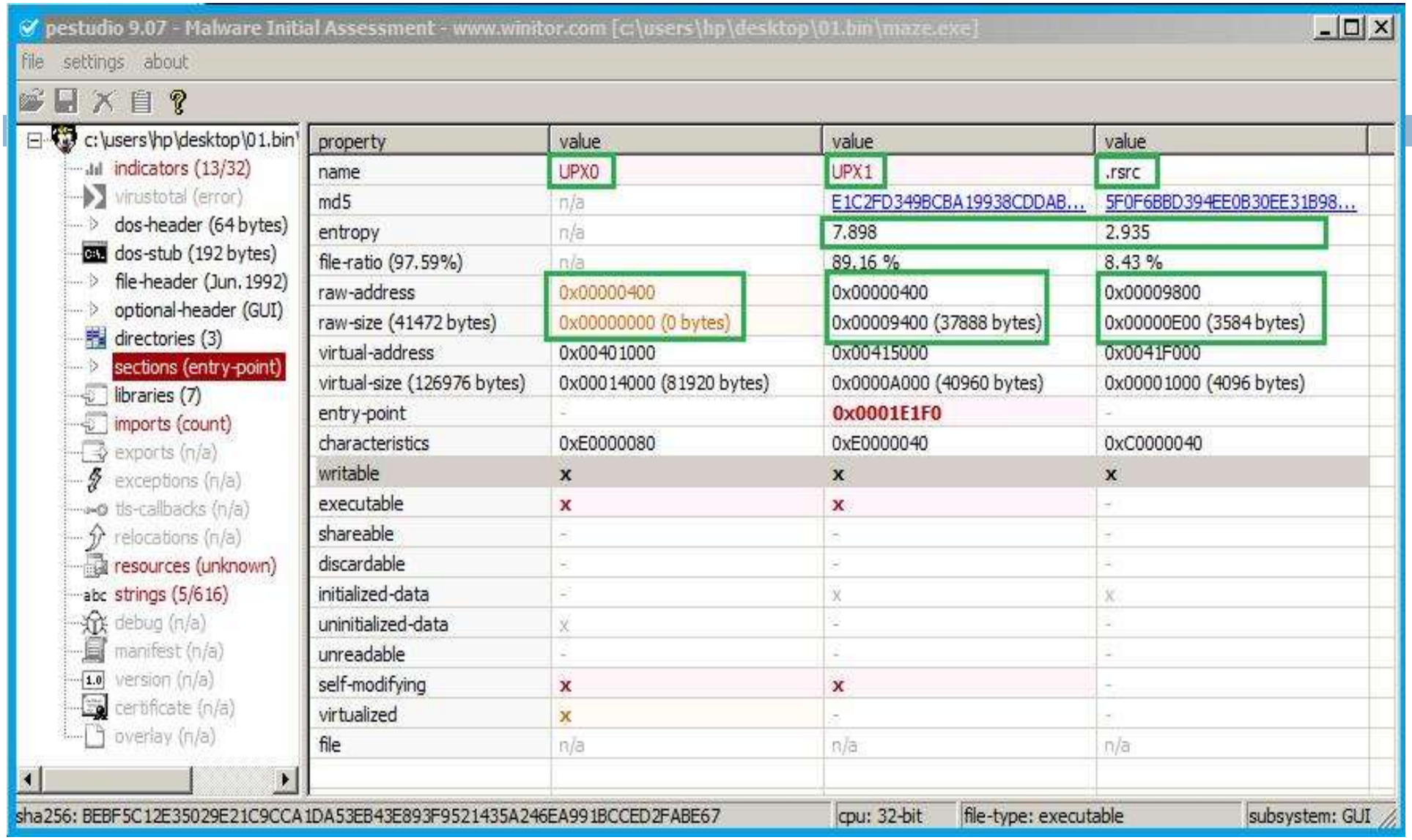
- Rely on **Indicators of Attack** instead of **Indicators of Compromise** alone:
  - Indicators of Attack (IOAs) are a way to take a proactive approach against fileless attacks. **IOAs do not focus on the steps of how an attack is being executed – instead, they look for signs that an attack may be in progress.**
- Indicators of Attack (IoA):
  - Process injection (e.g., into explorer.exe)
  - Unusual PowerShell execution
  - Privilege escalation attempts
  - Lateral movement across network
  - Credential dumping behavior
  - Suspicious parent-child process chains
- Indicators of Compromise:
  - Malicious file hashes (MD5, SHA256)
  - Suspicious IP addresses / domains
  - Registry changes (persistence keys)
  - Unexpected processes running
  - Modified system files
  - Log anomalies (failed logins, unusual access)

Feature	IoC (Indicators of Compromise)	IoA (Indicators of Attack)
Focus	Evidence of breach	Attacker behaviour
Timing	After attack	During attack
Detection Type	Signature-based	Behaviour-based
Example	File hash, IP	Process injection

- Employ Managed Threat Hunting:
  - Threat hunting for fileless malware is time-consuming and laborious work that requires the gathering and normalization of extensive amounts of data. Yet it is a necessary component in a defense that protects against fileless attacks, and for these reasons, the most pragmatic approach for the majority of organizations is to **turn their threat hunting over to an expert provider.**
  - **Managed threat hunting services** are on watch around the clock, proactively searching for intrusions, monitoring the environment, and recognizing subtle activities that would go unnoticed by standard security technologies.

Unpack indicators **Static:**

- High Entropy (Key Indicator)
- Suspicious Section Names:
  - Normal: .text, .data, .rdata
  - Packed:UPX0, UPX1.aspack, .themida
  - Random names
- Very Small Import Table
  - Few or no imports → suspicious
  - Often only: LoadLibrary ,GetProcAddress
- Entry Point in Weird Location
  - Entry point not in .text
  - Points to:
    - Last section
    - Unknown section
- Strings Are Missing or Garbage:
  - No readable strings like URLs, file paths
  - Indicates encrypted payload


pestudio 9.07 - Malware Initial Assessment - www.winitor.com [c:\users\hp\desktop\01.bin\maze.exe]
file settings about
c:\users\hp\desktop\01.bin
indicators (13/32)
virustotal (error)
dos-header (64 bytes)
dos-stub (192 bytes)
file-header (Jun.1992)
optional-header (GUI)
directories (3)
sections (entry-point)
libraries (7)
imports (count)
exports (n/a)
exceptions (n/a)
tls-callbacks (n/a)
relocations (n/a)
resources (unknown)
strings (5/616)
debug (n/a)
manifest (n/a)
version (n/a)
certificate (n/a)
overlay (n/a)
property value value value
name UPX0 UPX1 .rsrc
md5 n/a E1C2FD349BCBA19938CDDAB... 5F0F6BBD394EE0B30EE31B98...
entropy n/a 7.898 2.935
file-ratio (97.59%) n/a 89.16 % 8.43 %
raw-address 0x00000400 0x00000400 0x00009800
raw-size (41472 bytes) 0x00000000 (0 bytes) 0x00009400 (37888 bytes) 0x00000E00 (3584 bytes)
virtual-address 0x00401000 0x00415000 0x0041F000
virtual-size (126976 bytes) 0x00014000 (81920 bytes) 0x0000A000 (40960 bytes) 0x00001000 (4096 bytes)
entry-point - 0x0001E1F0 -
characteristics 0xE0000080 0xE0000040 0xC0000040
writable x x x
executable x x -
shareable - - -
discardable - - -
initialized-data - x x
uninitialized-data x - -
unreadable - - -
self-modifying x x -
virtualized x - -
file n/a n/a n/a
sha256: BEBF5C12E35029E21C9CCA1DA53EB43E893F9521435A246EA991BCCED2FABE67
cpu: 32-bit
file-type: executable
subsystem: GUI

Unpack indicators **Dynamic:**

- Memory Allocation & Execution Calls like:
  - VirtualAlloc
  - WriteProcessMemory
  - CreateProcess
- Self-Modifying Code: Code changes during execution
- Execution Redirection : Program jumps from unpacking stub to real code (Original Entry Point)

Code Injection and API Hooking

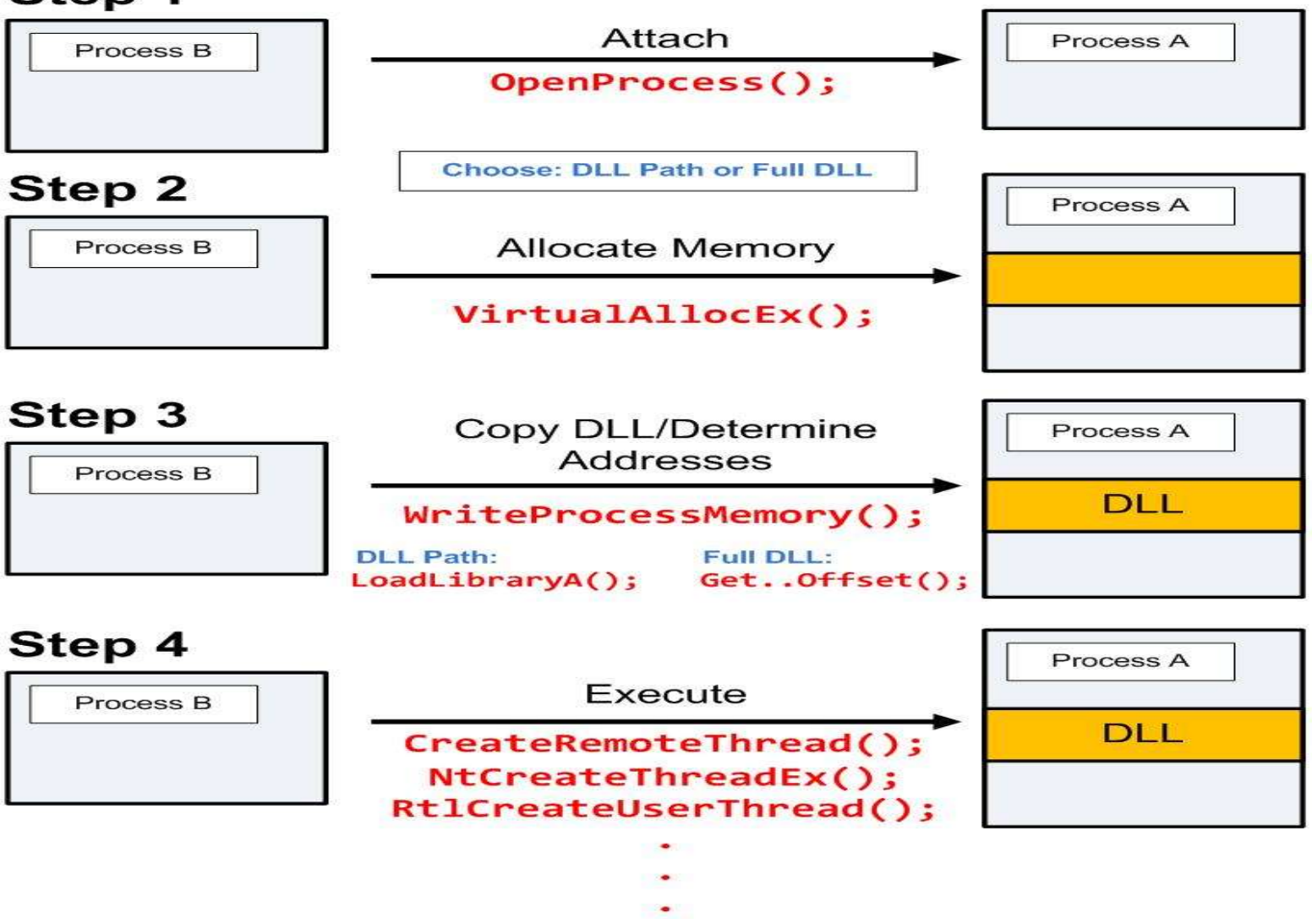
- API hooking is a technique that developers use for **manipulating the behavior of a system or an application.** With the help of **API hooking,** you can **intercept calls in a Windows application or capture information related to API calls.** Additionally, API hooking is one of the techniques that **antivirus and Endpoint Detection and Response solutions use for identifying malicious code.**
- What is the difference between injection and hooking?
  - **A hook procedure can act on each event it receives, and then modify or discard the event.** DLL injection is a technique used for running code within the address space of another process by forcing it to load a dynamic-link library.
- There are many ways you can implement API hooking. The three most popular methods are:
  - **DLL injection —** Allows you to **run** your **code inside a Windows process to perform different tasks**
  - **Code injection —** Implemented via the **WriteProcessMemory API used** for hiding custom code into another process
  - **Win32 Debug API toolset —** Provides you with full control over a debugged application, making it easy to manipulate the memory of a debugged process

Types of Hooking

DLL Injections

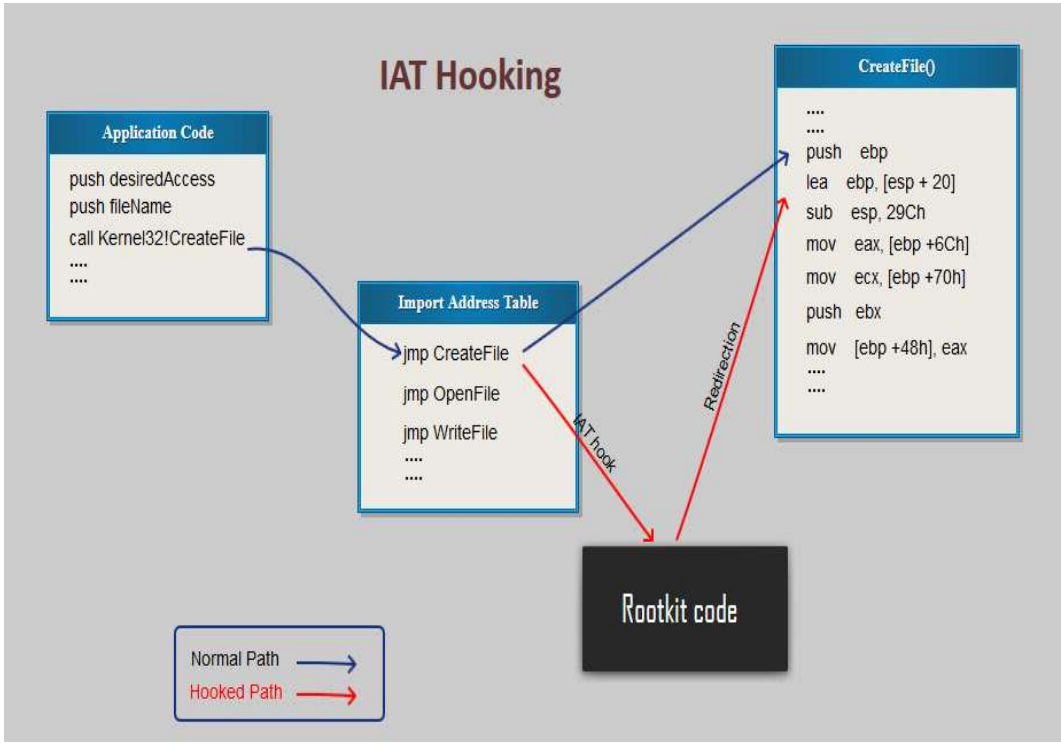
- SetWindowsHookEx:
  - It installs an application-defined hook procedure into a hook chain.
- We use it to install a hook procedure to monitor the system for certain types of events.
  - These events are associated either with a specific thread or with all threads in the same context as the calling thread. The most famous example implementation of this function is a keylogger application.
  - **For installing the hook, we require a malicious DLL which exports one or more functions. These functions will be called whenever the hooked events occur.**
  - We then create a program which loads the above DLL in memory using LoadLibrary and then call SetWindowsHookEx function.
  - The 1st parameter for function is the specific event which is to be hooked. In case of Keyloggers, the event name is WH_KEYBOARD. Other parameters are name of the DLL and the address of the exported method, which can be found using GetProcAddress.

Overview


Step 1
Process B
Attach
OpenProcess();
Process A
Choose: DLL Path or Full DLL
Step 2
Process B
Allocate Memory
VirtualAllocEx();
Process A
Step 3
Process B
Copy DLL/Determine Addresses
WriteProcessMemory();
DLL Path:
LoadLibraryA();
Full DLL:
Get..Offset();
Process A
DLL
Step 4
Process B
Execute
CreateRemoteThread();
NtCreateThreadEx();
RtlCreateUserThread();
.
.
.
Process A
DLL

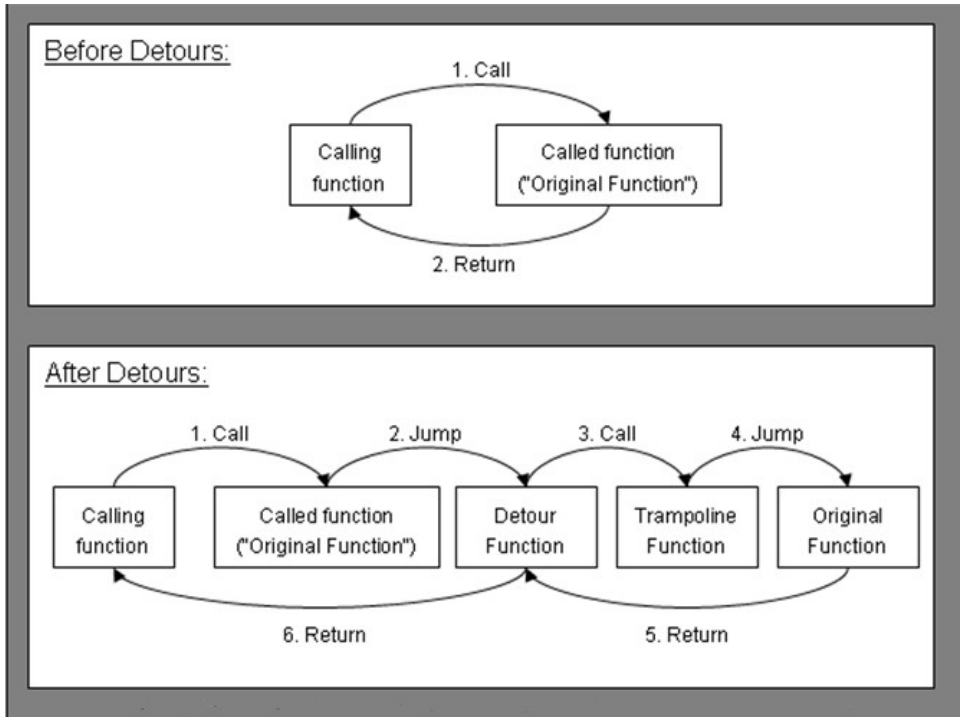
IAT Hooking

- Import Address Table (IAT) is an array of links representing the various DLLs imported by the PE loader during process initiation.
- IAT hooking is a technique of modifying the address of a particular DLL in the IAT with address of hook function. Before performing IAT hooking we must make sure that we are able to put the hook function in the user's address space through any of the DLL injection methods.
- IAT hooking will not be useful to us if the target program performs run-tie dynamic linking through LoadLibrary and GetProcAddress APIs to get the real address of each DLL functions.
- To get around this, hooking the GetProcAddress function would be the only solution but it will be a much tougher job.



Inline Hooking

- Inline Hooking is mostly seen in userland process than kernel mode processes.
- Typically, an inline function hook is implemented by overwriting the beginning of target function with an unconditional jump to a Detour function.
- Detour function calls a Trampoline function, which contains the overwritten bytes of the original target function, and then calls the target function.
- The target function returns to the detour function which finally gives control back to the source function. This whole process would appear more clear from the diagram below.



IDT Hooking

- Interrupt Descriptor Table (IDT) stored in IDT register contains pointer to Interrupt Service Routines (ISR).
- IDT hooking as the name suggest would modify the IDT entries to execute the hook function each time the interrupts are received.
- As each processor has a different IDT register, we make sure that the IDT entry we want to hook points to the same hooked ISR on all processor cores or else the hook will execute only a certain number of times.
- IDT register can be manipulated with the LIDT (Load IDT) and SIDT (Store IDT) instructions. SIDT will obtain the address of IDTR, and LIDT being a privileged instruction can be used to make changes to the IDTR. Sample program to perform IDT hooking can be referred from here.
- Global Descriptor Table (GDT) hooks are similar to IDT hooks. SGDT and LGDT instructions are used to modify the register contents. These descriptor structures are protected by Kernel Patch Protection as described earlier.

Sysenter Hooking

- System calls provide userland processes a way to request services from the kernel. The SYSENTER instructions (and equvialent SYSCALL on AMD) enable fast entry to the kernel, avoiding interrupt overhead. Sysenter is faster than the previous INT 0x2e only because it uses various Model Specific Registers (MSR) like SYSENTER_EIP, SYSENTER_ESP and SYSENTER_CS.
- To get more understanding on sysenter, like the significance of each MSR and how these are used to fetch the addresses, this FireEye blog would be a good reference. One important concept to note is that Sysenter is called in Ntdll.dll and it jumps to the value assigned in SYSENTER_EIP register which is also called as MSR-176h. That means for sysenter hooking, we have to modify the SYSENTER_EIP register.
- Modifications to MSRs are done using “wrmsr” instruction. The most easy bypass for Sysenter hooking would be to rewriting the register to its original value, however because KiFastCallEntry is not exported by ntoskrnl, getting the address could be tricky.

Using Memory Forensics for Malware Analysis

- Capturing and analyzing volatile data is essential for discovering important evidence. Making a RAM dump should become a standard operating procedure when acquiring digital evidence before pulling the plug and taking the hard drive out. Live Acquisition involves the capture of data from a system that is running when you encounter it.
- When to consider live capture:
  - Loss of data during shutdown
  - Pagefile set in registry to wipe at shutdown.
  - "Evidence eliminator" apps that remove data at shutdown.
  - Data not stored on disk (RAM contents, open ports, running processes, logged on users, etc.)
  - Encryption
  - Full Disk Encryption or open encrypted volumes
  - Cached passwords/passphrases in RAM
  - Volume of Data
  - Too much to image everything?
  - If you don't need it all…
  - Incident Response
  - Volatile data lost if you turn off the computer.
  - Suspect processes running only in RAM, not on disk.
  - Court or client-imposed business interruption restrictions
  - Kiosk/Internet Café
  - Maybe no hard drive, booted by CD and everything is in RAM.

Types of Evidence Available in Volatile Memory

Many types of evidence are available in computer's volatile memory that can be extracted by analyzing memory dumps. Volatile and ephemeral evidence types include:

- Running processes and services.
- Unpacked/decrypted versions of protected programs.
- System information (e.g., time elapsed since last reboot)
- Information about logged in users.
- Registry information
- Open network connections and ARP cache
- Remnants of chats, communications in social networks and MMORPG games
- Recent Web browsing activities including IE InPrivate mode and similar privacy-oriented modes in other Web browsers.
- Recent communications via Webmail systems
- Information from cloud services
- Decryption keys for encrypted volumes mounted at the time of the capture.
- Recently viewed images.
- Running malware/Trojans.